МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 3

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Вычисление геометрического центра: распараллеливание цикла и функциональная декомпозиция»

Работу выполнил
Студент гр. 4404
Зарипов А.Э.

Казань, 2026

1. Цель работы

Изучение двух основных подходов к декомпозиции параллельных задач: декомпозиции по данным (распараллеливание цикла, data decomposition) и функциональной декомпозиции (functional decomposition). В качестве практической задачи выбрано вычисление геометрического центра множества точек в трёхмерном пространстве: среднее арифметическое координат по осям x, y и z. Реализованы две версии программы на языке Go и проведено их сравнение по времени выполнения и корректности результата.

Задача 5 методических указаний «Параллельное программирование на OpenMP» выполняется на языке Go, поэтому конструкции OpenMP моделируются стандартными средствами языка: горутины и runtime.GOMAXPROCS соответствуют команде потоков, распараллеливание цикла реализуется через распределение непрерывных блоков точек между горутинами с ручным объединением частичных сумм (аналог редукции), а функциональная декомпозиция — через три параллельно выполняющиеся горутины, каждая из которых суммирует значения по одной из координат (аналог конструкции sections в OpenMP). sync.WaitGroup используется как неявный барьер в конце параллельной области.

Представляемые материалы: работающая программа (корректность её выполнения подтверждена скриншотами запусков, рисунки 1–4), исходный код в публичном репозитории GitHub и отчёт со скриншотами по ходу работы, ответами на поставленные вопросы и выводами.

2. Задание

Задача 5. Напишите программу, которая читает из файла координаты точек в 3D пространстве (x, y, z) и вычисляет геометрический центр, который есть среднее по x, y и z. Напишите две версии программы: одну с распараллеливанием цикла, другую с функциональной декомпозицией.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу; каждый запуск, о котором говорится в задаче (разные версии программы), подтверждается скриншотом с видимой командой запуска и полным выводом программы.

3. Ход работы

3.1. Математическая постановка задачи

Геометрический центр множества из N точек (x_i, y_i, z_i) вычисляется как среднее арифметическое координат по каждой из осей:

center_x = (Σ x_i) / N;

center_y = (Σ y_i) / N;

center_z = (Σ z_i) / N.

Таким образом, задача сводится к вычислению трёх сумм по всем точкам и делению каждой суммы на N. Исходные данные представляют собой текстовый файл, каждая строка которого содержит три числа — координаты x, y и z одной точки.

3.2. Работа с входным файлом: генерация и чтение точек

Программа поддерживает отдельный режим генерации входного файла командой вида `go run . gen <task_size> [file]`. Генератор создаёт детерминированные точки с координатами (i, 2·i, 3·i) для i = 0 … N-1. Для таких точек ожидаемый геометрический центр можно вычислить аналитически:

Σ i = N·(N-1)/2 $\Rightarrow$ center_x = (N-1)/2;

Σ 2·i = N·(N-1) $\Rightarrow$ center_y = N-1;

Σ 3·i = 3·N·(N-1)/2 $\Rightarrow$ center_z = 3·(N-1)/2.

Для N = 2 000 000 ожидаемый геометрический центр равен (999999.5, 1999999.0, 2999998.5).

Фрагмент кода (lab3/main.go, функция generatePoints):

```
func generatePoints(filename string, n int) error {
    f, err := os.Create(filename)
    if err != nil { return err }
    defer f.Close()
    w := bufio.NewWriterSize(f, 1<<20)
    for i := 0; i < n; i++ {
        fmt.Fprintf(w, "%d %d %d\n", i, 2*i, 3*i)
    }
    return w.Flush()
}
```

Чтение точек выполняется функцией readPoints: файл открывается через bufio.Scanner, каждая строка разбирается функцией strings.Fields, первые три токена преобразуются в float64 через strconv.ParseFloat. Пустые строки и строки, начинающиеся с «#», пропускаются. Программа читает ровно столько точек, сколько указано в task_size; если в файле точек меньше — возвращается ошибка.

Фрагмент кода (lab3/main.go, функция readPoints):

```
func readPoints(filename string, need int) ([]float64, []float64, []float64,
error) {
    f, err := os.Open(filename)
    if err != nil { return nil, nil, nil, err }
    defer f.Close()
    sc := bufio.NewScanner(f)
    xs := make([]float64, need)
    ys := make([]float64, need)
    zs := make([]float64, need)
    count := 0
    for sc.Scan() {
        line := strings.TrimSpace(sc.Text())
        if line == "" || strings.HasPrefix(line, "#") { continue }
        fields := strings.Fields(line)
        x, _ := strconv.ParseFloat(fields[0], 64)
        y, _ := strconv.ParseFloat(fields[1], 64)
        z, _ := strconv.ParseFloat(fields[2], 64)
        xs[count], ys[count], zs[count] = x, y, z
        count++
        if count == need { break }
    }
    if count < need { return nil, nil, nil, fmt.Errorf(...) }
    return xs, ys, zs, nil
}
```

Команда генерации входного файла и результат её работы показаны на рисунке 1. Генерация двух миллионов точек занимает порядка одной секунды, после чего файл points.txt готов для использования в основных запусках.

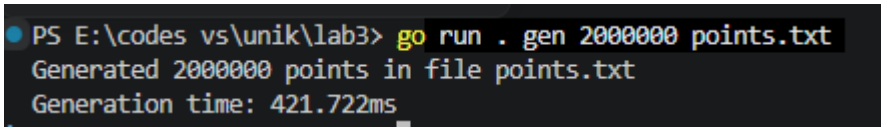


Рисунок 1 — Генерация входного файла на 2 000 000 точек

3.3. Версия 1. Распараллеливание цикла (декомпозиция по данным)

В первой версии задачи декомпозиция выполняется по данным: множество точек разбивается на непересекающиеся непрерывные блоки, каждый блок обрабатывается отдельной горутиной. Каждая горутина вычисляет локальные частичные суммы по x, y и z только для своего блока точек, не взаимодействуя с другими горутинами. После завершения всех горутин частичные суммы объединяются в глобальные. Такой подход соответствует директиве OpenMP `#pragma omp parallel for reduction(+:sum_x, sum_y, sum_z)`.

Фрагмент кода (lab3/main.go, функция computeLoopParallel):

```
chunkSize := (n + threads - 1) / threads
for t := 0; t < threads; t++ {
    start := t * chunkSize
    if start >= n { break }
    end := start + chunkSize
    if end > n { end = n }
    active++
    wg.Add(1)
    go func(id, s, e int) {
        defer wg.Done()
        var sx, sy, sz float64
        for i := s; i < e; i++ {
            sx += xs[i]; sy += ys[i]; sz += zs[i]
        }
        partials[id] = vec3{x: sx, y: sy, z: sz}
    }(t, start, end)
}
wg.Wait()
// объединение частичных сумм partials в глобальные sx, sy, sz
```

В этой версии степень параллелизма легко масштабируется с числом потоков: при увеличении числа потоков число одновременно обрабатываемых блоков растёт, и производительность повышается до ограничений, накладываемых пропускной способностью памяти.

На рисунке 2 показан запуск первой версии программы (mode := "loop") с 8 потоками и 2 000 000 точек, прочитанных из файла points.txt. Вывод содержит

имя файла, число активных рабочих, время чтения файла, время вычислений и полученный геометрический центр, совпадающий с ожидаемым значением.

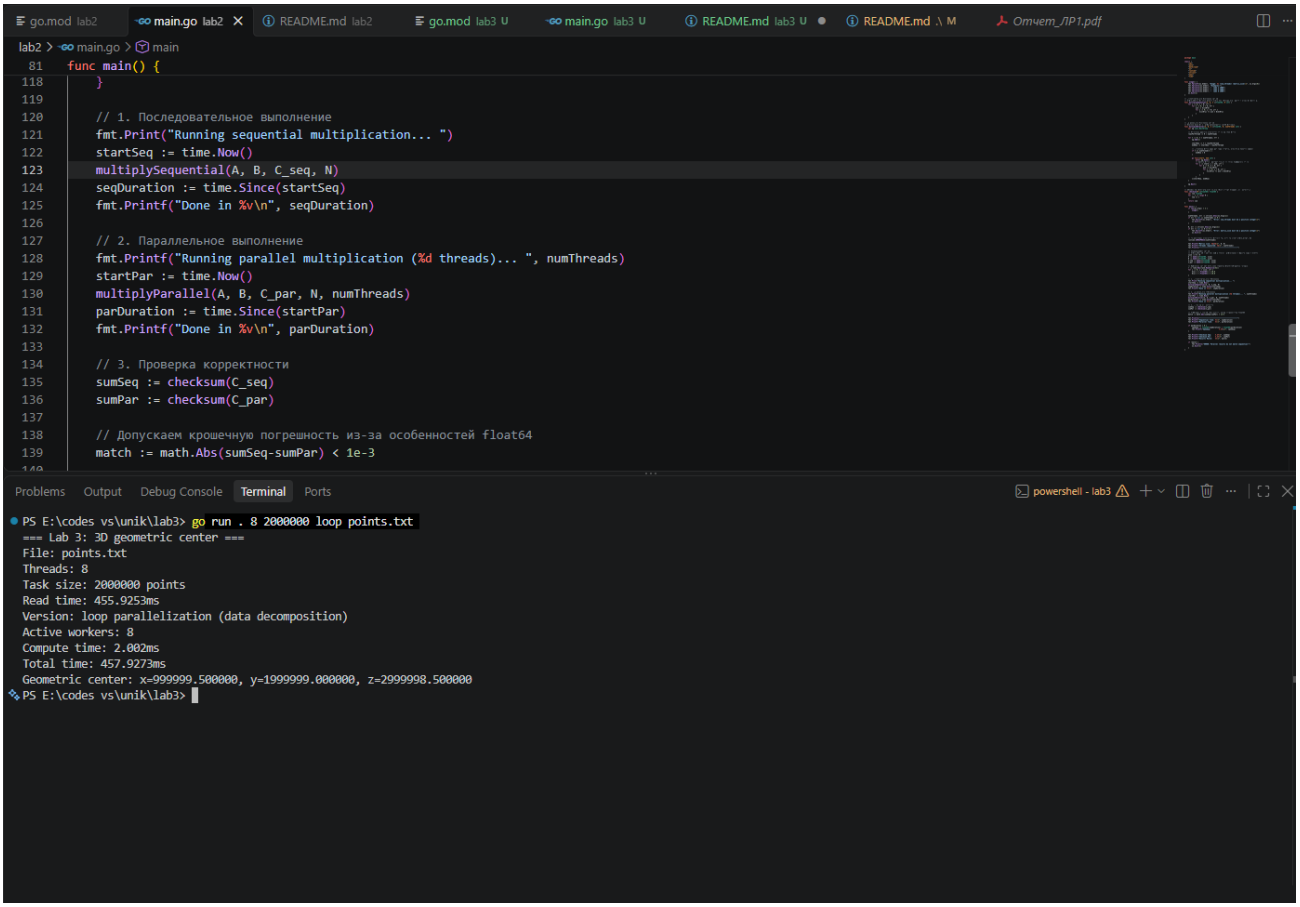


Рисунок 2 — Версия 1: распараллеливание цикла (mode := "loop")

3.4. Версия 2. Функциональная декомпозиция

Во второй версии задачи декомпозиция выполняется не по данным, а по функциям: каждая горутина отвечает за суммирование значений по одной из координат. Запускаются три горутины — sumX, sumY, sumZ — каждая проходит по всему массиву своей координаты и вычисляет сумму. Этот подход соответствует конструкции `#pragma omp sections` в OpenMP, где каждая секция выполняется отдельным потоком. Число вычислительных потоков здесь жёстко ограничено тремя (по числу осей), поэтому увеличение числа потоков сверх 3 не даёт прироста производительности в вычислительной части программы.

Фрагмент кода (lab3/main.go, функция computeFunctional):

```
    var sx, sy, sz float64
```

```
var wg sync.WaitGroup
wg.Add(3)
go func() { defer wg.Done(); sx = sumSlice(xs) }()   // sumX
go func() { defer wg.Done(); sy = sumSlice(ys) }()   // sumY
go func() { defer wg.Done(); sz = sumSlice(zs) }()   // sumZ
wg.Wait()
```

Преимущество функциональной декомпозиции — простота и минимальное число точек синхронизации (один общий барьер в конце). Недостаток — фиксированная степень параллелизма (три потока) и последовательный доступ к памяти в каждой горутине по одному массиву, что эффективно использует кэш, но ограничивает ускорение.

На рисунке 3 показан запуск второй версии программы (mode := "func") с теми же исходными данными из файла points.txt. Геометрический центр совпадает с результатом первой версии, но время вычислений может отличаться, так как используются только 3 параллельные горутины.

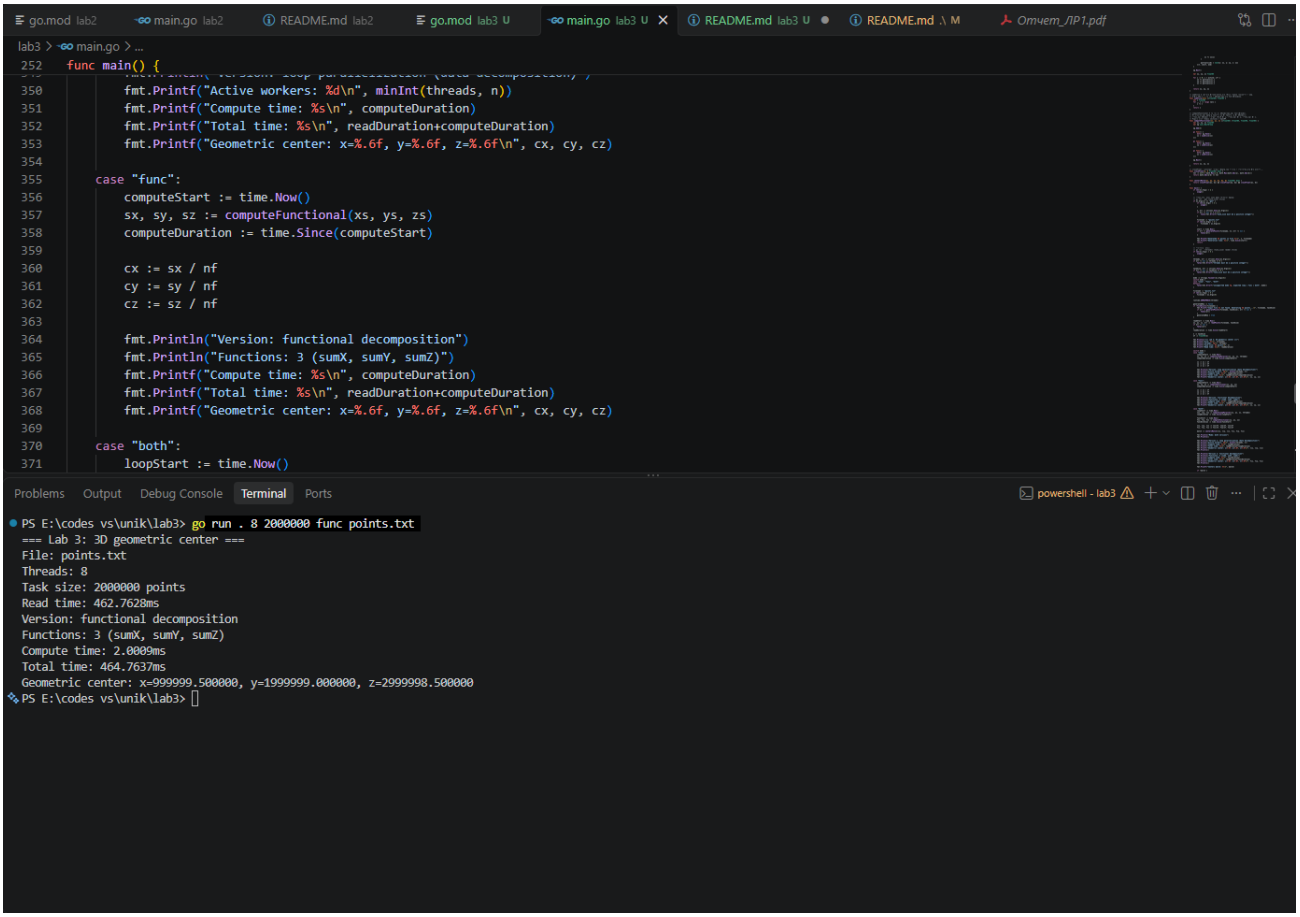


Рисунок 3 — Версия 2: функциональная декомпозиция (mode := "func")

3.5. Сравнение обеих версий

Режим "both" выполняет обе версии программы подряд и сравнивает полученные геометрические центры. Совпадение центров с заданной точностью подтверждает корректность обеих реализаций. Кроме того, поскольку точки были сгенерированы самой программой, в конце вывода печатается ожидаемый аналитический центр — ещё один независимый способ проверки корректности.

На рисунке 4 показан запуск в режиме сравнения (mode := "both"). Вывод содержит имя файла, результаты обеих версий, признак совпадения центров (Centers match: true) и ожидаемый центр для сгенерированных точек.

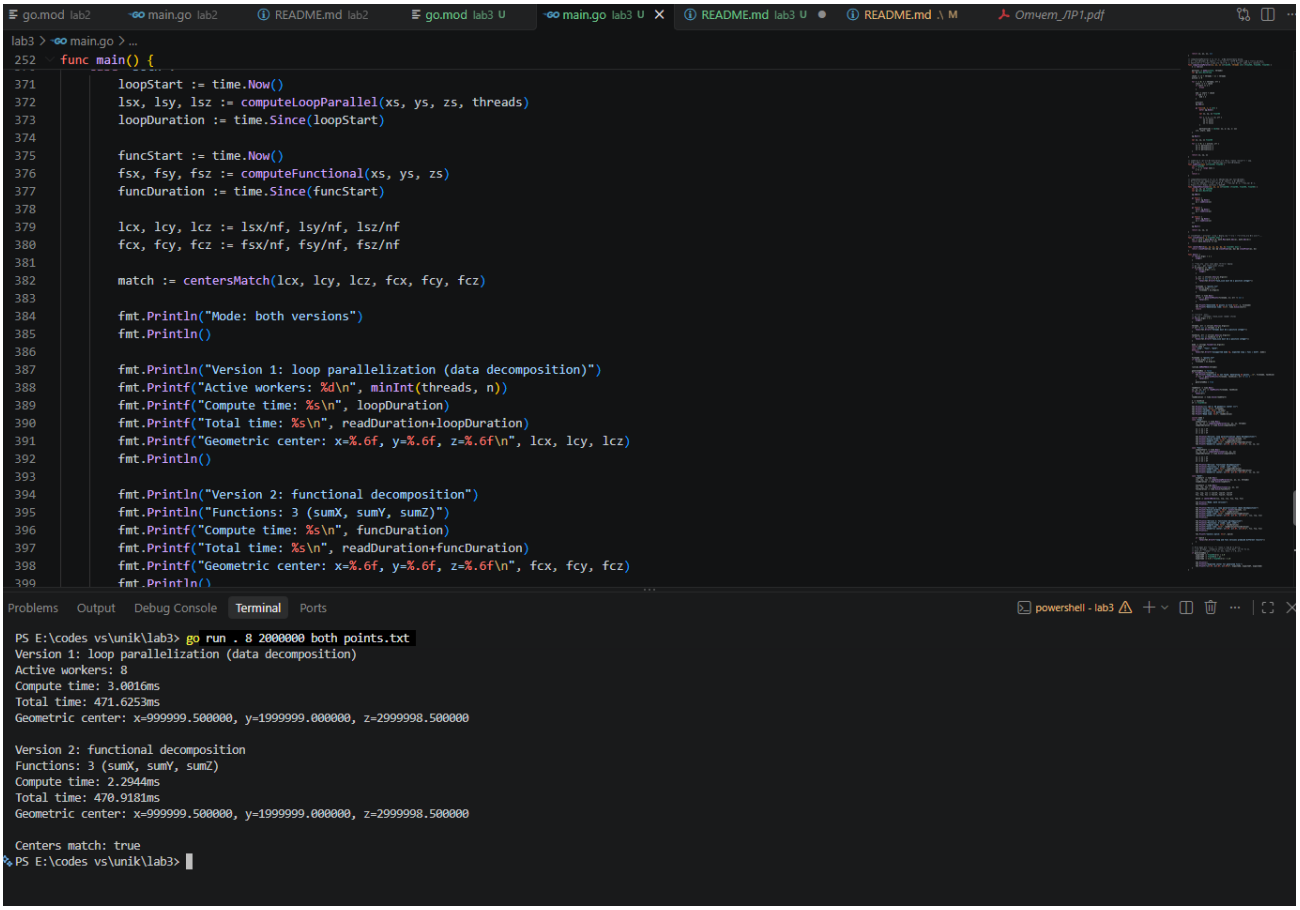


Рисунок 4 — Сравнение обеих версий (mode := "both")

Таблица 1 — Сравнение подходов к распараллеливанию

Параметр	**Версия 1: loop**	**Версия 2: func**
Тип декомпозиции	по данным	по функциям
Аналог в OpenMP	parallel for + reduction	parallel sections

Число рабочих потоков	равно numThreads	фиксированно 3
Масштабируемость	хорошая	ограничена тремя функциями
Точки синхронизации	один барьер + объединение сумм	один барьер
Корректность	подтверждена	подтверждена

4. Ответы на вопросы, поставленные в задачах

4.1. Вопрос задачи 5: сравнение двух версий программы

Явный вопрос в задаче 5 не поставлен, однако требуется продемонстрировать две версии программы и убедиться в их корректности. По результатам запусков (рисунки 2–4):

— обе версии вычисляют одинаковый геометрический центр, совпадающий с ожидаемым аналитическим значением (Centers match: true);

— версия с распараллеливанием цикла (loop) масштабируется с числом потоков и обычно быстрее при большом числе потоков (8 и более);

— версия с функциональной декомпозицией (func) использует только 3 рабочих горутины и не даёт прироста при увеличении числа потоков сверх 3, но имеет меньше накладных расходов на синхронизацию и проще в реализации.

Выбор подхода зависит от структуры задачи: если вычисления однородны по всем данным (как в данной задаче), эффективнее распараллеливание цикла; если задача состоит из качественно разных независимых подзадач, естественнее функциональная декомпозиция.

5. Выводы

1. Реализованы две версии программы вычисления геометрического центра множества 3D-точек на языке Go: с распараллеливанием цикла (декомпозиция по данным) и с функциональной декомпозицией (три параллельные функции

sumX, sumY, sumZ). Программа читает координаты точек из текстового файла; для подготовки файла встроен генератор точек (команда gen).

2. Обе версии вычисляют одинаковый результат, совпадающий с аналитически ожидаемым значением для сгенерированных точек. Корректность подтверждена сравнением геометрических центров (Centers match: true) на рисунках 2–4.

3. Версия с распараллеливанием цикла хорошо масштабируется с числом потоков и является предпочтительной для задач однородной обработки большого массива данных. Версия с функциональной декомпозицией имеет фиксированный параллелизм (3 потока), проще реализуется и подходит, когда задача естественно разбивается на независимые разнородные подзадачи.

4. В обеих версиях гонки данных отсутствуют, поскольку каждая горутина пишет только в свою область памяти, а объединение результатов происходит после барьера sync.WaitGroup. Это позволило обойтись без мьютексов и атомарных операций.